

データ構造とアルゴリズム II

《演習-1》 分割統治法と貪欲法

課題

- 分割統治法
 - 最大部分配列問題
- 語彙探索木
 - Dijkstra法

提出物

- 設計図：フローチャート、NSチャート、HCP、PAD、…
- C言語プログラムコード
 - 提供するプログラム・スケルトンを埋めて作成すること
- 実行結果

提出期限と採点

提出期限

- 5月14日 17:00

課題の採点

- 各 10 点満点
 - 設計図 2 点
 - プログラムコード 4 点
 - 実行結果 2 点
 - 解説、説明等 2 点

データ構造とアルゴリズム II

《演習》
最大部分配列問題

プログラムの全体構造

作成するプログラム

- MSP.c : 最大部分配列の和を貪欲法で求めるプログラム
 - 設計図とコードを作成

用意されているプログラム

- MSP.h : 作成する関数の仕様
- Data.c : テスト用のデータ
 - 動作テストはこのデータで行い、レポートの作成時は数値を各自変更して正常動作を確認すること

部分配列の表現方法

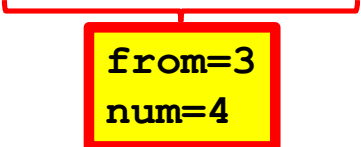
2個の符号なし整数で表す

●**from** : 部分配列の先頭の添え字

●**num** : 部分配列の個数 (長さ)

例 : **data** []

<i>0</i>	<i>1</i>	<i>2</i>	<i>3</i>	<i>4</i>	<i>5</i>	<i>6</i>	<i>7</i>	<i>8</i>	<i>9</i>	<i>10</i>	<i>11</i>
-1	2	-3	5	-2	1	3	-2	-2	-3	6	-1



from=3
num=4

MSP.c

```
extern unsigned num;
extern int data[];

int main(void){
    printf("%d\\n", maxSubarray(0, num));
    return 0;
}

int maxSubarray(unsigned from, unsigned num){
    /* stop recursive call */
    /* divide */
    /* conquer */
    /* combine */
}
```

配列の要素数
配列データ } 入力データ
(Data.cの中で定義)

配列全体

MSP.c : **/* stop recursive call */**

部分配列の個数が1のとき (**num==1**)

- **data[from]** の値を返す

部分配列の個数が2のとき (**num==2**)

- **data[from], data[from+1], data[from]+data[from+1]**
の中でもっとも大きな値を返す

その他の場合 (**num>=3**)

- divide,conquere,combineに進む

MSP.c : /* divide */

- サイズが奇数でも偶数でもよいコードを作成する
 - if文などで場合分けしないこと
 - C言語の整数除算は端数を切り捨てる

C言語	偶数	奇数
numA	10	11
fromA	a	a
mid=num/2	5	5
numA1=mid	5	5
fromA1	a	a
numA2=numA-numA1	5	6
fromA2=fromA+numA-1	a+4	a+4

「半分に」分割

MSP.c : **`/* conquer */`**

- 1.前半部分に対して再帰呼び出し
 - ・ 最大部分配列の和 **max1** を得る
- 2.後半部分に対して再帰呼び出し
 - ・ 最大部分配列の和 **max2** を得る
- 3.前半部分と後半部分にまたがる最大部分配列 **max3** の和を求める
 - 3.1 境界から左へ最大のと **max3L**を求める
 - 3.2 境界から右へ最大のと **max3R**を求める
 - 3.3 **max3=max3L+max3R**

MSP.c : **/* combine */**

- conquerで求めた**max1, max2, max3**の中の最大値**max**を求める
- **max**が**maxSubarray**関数の戻り値である

コンパイル & 実行

```
gcc -o testMSP MSP.c Data.c  
./testMSP
```

データ構造とアルゴリズム II

《演習》
ダイクストラ法

ダイクストラ法の改良

$d_0 = 0, d_i = \infty (i > 0), U = \{\text{すべての頂点}\}$

while $U \neq \emptyset$ do

U から距離が最小な頂点 x_m を取り出す

for each $x_i \in \{x_m \text{の近隣頂点}\}$

$d_i \leftarrow \min\{d_i, d_m + w(x_m, x_i)\}$

U の中の x_m の近隣頂点

- U から既に取り除いた頂点は最適解が決まっているので、
 U に残っている頂点についてのみ局所的最適解を求めればよい

ヒープを用いた高速化

```
 $d_0 = 0, d_i = \infty (i > 0)$ , ヒープ $H$ の構築  
while  $U \neq \emptyset$  do  
    頂点 $x_m \leftarrow H$ から根ノードを取り出す  
    for each  $x_i \in \{H \text{ 中の } x_m \text{ の近隣頂点}\}$   
         $d_i \leftarrow \min\{d_i, d_m + w(x_m, x_i)\}$   
        if  $d_i$  が変更 then  $H$ を更新する
```

ヒープ H

- ・ ノードには頂点のID(x_i の i)を格納する
- ・ 既知の準最適距離 (d_i) が小さい順にソートされている

配列を用いたヒープの実装

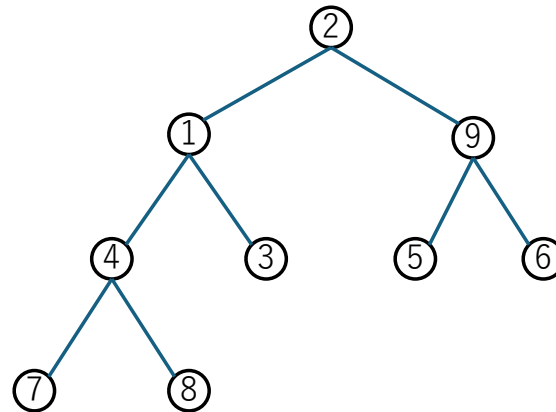
```
typedef struct {  
    unsigned num;    ノードの個数  
    unsigned val[]; データ  
} HEAP;
```

ノード $H.val[i]$ の...

- ・ 親ノード : $H.val[(i-1)/2]$
- ・ 左子ノード : $H.val[2*i+1]$
- ・ 右子ノード : $H.val[2*i+2]$

頂点	x_0	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9
d_i	0	3	1	∞	6	∞	∞	6	∞	∞

done!



```
H.num=9;  
H.val[]={  
    2,  
    1, 9,  
    4, 3, 5, 6,  
    7, 8  
};
```

ヒープの生成と削除

戻り値／引数がポインタ
であることに注意！

HEAP *newHeap(unsigned num)

- ノード数numのヒープを作成する
【使用例】 `HEAP h=newHeap(10);`
 - ノード数10のヒープを作成し、変数hに格納する

void freeHeap(HEAP *heap)

- ヒープheapを削除する
【使用例】 `freeHeap(h);`
 - 変数hに格納されているヒープを削除する

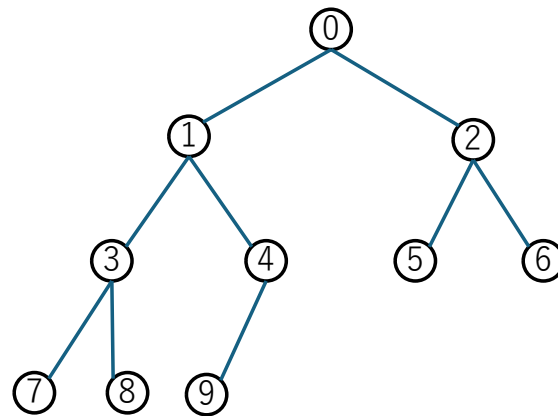
生成したヒープは
使わなくなったら必ず
消去すること！

ヒープの構築

- $d_0 = 0, d_i = \infty (i > 0)$ なのでノード i は頂点 x_i で初期化してよい

```
HEAP *heap = newHeap(numN); ヒープの生成
for ( unsigned i = 0; i < numN; i++ )
    heap->val[i] = i; ヒープの初期化
```

例：numN=10



```
H.num=10;
H.val[]={
    0,1,2,3,4,
    5,6,7,8,9};
```

ヒープから根ノードを取り出す

unsigned removeRoot(HEAP *heap)

- 戻り値は取り除いた根ノードに格納されていた x_i
- ヒープソートで、ソート終了後に1個ずつデータを取り出す時と同じ操作
 1. 最後尾のノードを根ノードに移動
 2. 根ノードをヒープ化する (heapify() 関数の呼び出し)

ヒープ化関数

void heapify(HEAP *heap, unsigned pos)

- ノード pos (頂点 x_p) でヒープ化する
 1. ノード pos 、左子ノード posL (頂点 x_L)、右子ノード posR (頂点 x_R) の距離 d_i が、ヒープ条件を満たすようにする
 2. もし x_p が (どちらかの) 子ノードに移動すれば、その子ノードでヒープ化する (再帰呼び出し)

☆ヒープソートを参照

ヒープの更新

void changeHeap(HEAP *heap, unsigned pos)

- ノード pos (頂点 x_p) の距離 d_p が小さくなったときにヒープを更新 (修正) するための関数
 1. ノード pos 、親ノード pos_X (頂点 x_X)、兄弟ノード pos_B (頂点 x_B) の距離 d_i が、ヒープ条件を満たすようにする
 2. もし x_p が親ノードに移動すれば、親ノードで再帰呼び出しする

☆`heapify`関数と異なり、根ノードの方に操作が進む

プログラムの全体構造

- Dijkstra.c: ダイクストラ法で x_0 から各頂点までの最小距離を求めるプログラム
- Dijkstra.h: 隣接行列とノード数の仕様
- Heap.c: ヒープ操作ライブラリ
 - newHeap関数、freeHeap関数、removeRoot関数: 作成済
 - heapify関数、chngeHeap関数: 演習で作成する
- Heap.h: データ構造と関数の仕様
- Data.c: テスト用のデータ
 - 動作テストはこのデータで行い、レポートの作成時は数値を各自変更して正常動作を確認すること

コンパイル & 実行

```
gcc -o testDijkstra Dijkstra.c Data.c  
./testDijkstra
```